

API SECURITY RISK ANALYSIS REPORT

Modern SaaS Security Audit | 2025
APIs Tested: JSONPlaceholder · ReqRes

Report Type	API Security Risk Analysis — Read-Only Audit
Scope	Public / Demo APIs: JSONPlaceholder, ReqRes
Classification	Internal — Confidential
Methodology	OWASP API Security Top 10 (2023 Edition)
Prepared By	AppSec Analyst / Security Consulting Team
Date	May 2025
Risks Identified	8 findings (4 High · 3 Medium · 1 Low)

Scope & Ethics Statement

This report is based entirely on READ-ONLY analysis of public demo APIs that are explicitly provided for testing and education. No exploitation, bypass, or denial-of-service testing was performed. All findings are presented for educational and awareness purposes only.

APIs tested: jsonplaceholder.typicode.com | reqres.in

1. Executive Summary

APIs are the backbone of modern SaaS applications, mobile platforms, and service integrations. Unlike traditional web application vulnerabilities, API security issues are harder to detect through automated scanning and require analytical reasoning about authentication flows, data exposure, and access control logic.

This report presents the findings of a read-only security audit conducted against two widely-used public demo APIs: JSONPlaceholder and ReqRes. These APIs represent common patterns found in real production SaaS backends, making them ideal study environments for identifying API security risk patterns.

8 Total Findings	4 High Severity	3 Medium Severity	1 Low Severity
---------------------	--------------------	----------------------	-------------------

Key Findings Summary

ID	Finding	Severity	OWASP Reference
RISK-01	Unauthenticated Access to All Endpoints	HIGH	API1:2023
RISK-02	Excessive Data Exposure in User Object	HIGH	API3:2023
RISK-03	No Rate Limiting on Any Endpoint	HIGH	API4:2023
RISK-04	Unauthenticated Write Operations (POST / PUT / DELETE)	HIGH	API5:2023
RISK-05	CORS Wildcard Allows Any Origin	MEDIUM	API8:2023
RISK-06	Missing Security Headers in All Responses	MEDIUM	API8:2023
RISK-07	User Enumeration via Predictable Sequential IDs	MEDIUM	API1:2023
RISK-08	No Input Validation or Schema Enforcement on POST	LOW	API6:2023

2. APIs Under Analysis

2.1 JSONPlaceholder

JSONPlaceholder (jsonplaceholder.typicode.com) is a free, open REST API service created for prototyping and testing. It provides fake data across 6 resource types: posts, comments, albums, photos, todos, and users. It is widely used in frontend tutorials, making its patterns representative of real API designs.

Base URL	https://jsonplaceholder.typicode.com
Resources	/posts (100), /comments (500), /albums (100), /photos (5000), /todos (200), /users (10)
Auth Required	None
Documentation	https://jsonplaceholder.typicode.com/guide/
Methods Tested	GET, POST, PUT, PATCH, DELETE

2.2 ReqRes

ReqRes (reqres.in) is a hosted REST API service specifically designed for frontend developers to test HTTP requests. It simulates a real user management system with login, registration, and user CRUD endpoints, making it ideal for analyzing authentication and authorization patterns.

Base URL	https://reqres.in/api
Resources	/users, /login, /register, /unknown
Auth Required	Token-based (simulated)
Documentation	https://reqres.in/
Methods Tested	GET, POST (login, register)

3. Methodology

This audit followed a structured, documentation-driven approach aligned with the OWASP API Security Top 10 (2023 Edition). All testing was read-only and non-destructive. The following methodology was applied:

- 1. Documentation Review — Analyzed official API documentation to understand intended behavior, authentication model, and data schema
- 2. Endpoint Enumeration — Identified all available resources and HTTP methods supported by each API
- 3. Authentication Analysis — Tested each endpoint with and without authentication tokens to assess access control enforcement
- 4. Header Inspection — Reviewed all request and response headers for security posture, information leakage, and missing protections
- 5. Response Analysis — Examined response bodies for excessive data exposure, sensitive field leakage, and error message verbosity
- 6. Risk Scoring — Assigned severity (High / Medium / Low) based on exploitability, data sensitivity, and business impact
- 7. Remediation Mapping — Mapped each finding to concrete, actionable developer guidance

OWASP API Security Top 10 — 2023

API1 — Broken Object Level Authorization	API6 — Unrestricted Access to Sensitive Business Flows
API2 — Broken Authentication	API7 — Server Side Request Forgery
API3 — Broken Object Property Level Auth	API8 — Security Misconfiguration
API4 — Unrestricted Resource Consumption	API9 — Improper Inventory Management
API5 — Broken Function Level Authorization	API10 — Unsafe Consumption of APIs

4. Detailed Risk Findings

The following section presents each identified risk in full detail, including evidence from actual API responses, business impact assessment, and step-by-step remediation guidance.

RISK-01 Unauthenticated Access to All Endpoints			HIGH
API Tested	JSONPlaceholder	OWASP Category	API1:2023
Endpoint(s)	GET /users, /posts, /comments, /todos		

Description

Every endpoint on JSONPlaceholder is fully accessible without any authentication token, API key, session cookie, or credential of any kind. Any anonymous HTTP client on the internet can retrieve all user records, posts, comments, and to-do items with a single GET request.

Evidence — Request / Response Sample

```
GET https://jsonplaceholder.typicode.com/users
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8

Response (excerpt):
[ { "id": 1, "name": "Leanne Graham", "username": "Bret",
  "email": "Sincere@april.biz",
  "address": { "street": "Kulas Light", "city": "Gwenborough", ... },
  "phone": "1-770-736-0988 x56442",
  "website": "hildegard.org",
  "company": { "name": "Romaguera-Crona", ... } },
... 9 more user objects ]

Authentication header sent: NONE
Rate-limit headers in response: NONE
```

Business Impact

Impact Analysis

In a production SaaS equivalent, this means any actor — competitor, bot, or attacker — can enumerate all users, extract PII (names, emails, addresses, phone numbers), and build targeted attack lists without leaving any audit trail.

Remediation Steps

1. Require Bearer token or API key on every non-public endpoint
2. Implement OAuth 2.0 / OpenID Connect for user-facing APIs
3. Return 401 Unauthorized for requests missing valid credentials
4. Log and alert on unauthenticated access attempts

RISK-02 Excessive Data Exposure in User Object			HIGH
API Tested	JSONPlaceholder	OWASP Category	API3:2023
Endpoint(s)	GET /users/{id}		

Description

The /users endpoint returns far more data than any client UI typically needs. The full user object exposes home address (street, suite, city, zip, geo-coordinates), personal phone number, personal website, and full company details including internal company catch-phrase strings — none of which should be part of a standard public API response.

Evidence — Request / Response Sample

```
GET https://jsonplaceholder.typicode.com/users/1
HTTP/1.1 200 OK

{
  "id": 1,
  "name": "Leanne Graham",
  "username": "Bret",
  "email": "Sincere@april.biz",
  "address": {
    "street": "Kulas Light", "suite": "Apt. 556",
    "city": "Gwenborough", "zipcode": "92998-3874",
    "geo": { "lat": "-37.3159", "lng": "81.1496" }
  },
  "phone": "1-770-736-0988 x56442",
  "website": "hildegard.org",
  "company": { "name": "Romaguera-Crona",
    "catchPhrase": "Multi-layered client-server neural-net",
    "bs": "harness real-time e-markets" }
}
```

Business Impact

Impact Analysis

In a real system, exposing home address, geo-coordinates, and phone in an API response violates GDPR, CCPA, and most data minimization principles. Scrapers can harvest this data at scale. Attackers can use geo-coordinates for physical targeting.

Remediation Steps

- 5. Apply response field filtering — only return fields the client actually needs

6. Create separate API response DTOs (Data Transfer Objects) per consumer role
7. Never include internal fields (geo-coordinates, company bs/catchPhrase) in public responses
8. Implement field-level authorization: authenticated users see more than guests

RISK-03 No Rate Limiting on Any Endpoint			HIGH
API Tested	JSONPlaceholder + ReqRes	OWASP Category	API4:2023
Endpoint(s)	All endpoints		

Description

Neither JSONPlaceholder nor ReqRes returns any rate-limiting headers (X-RateLimit-Limit, X-RateLimit-Remaining, Retry-After) in their responses. There is no evidence of request throttling, connection limiting, or IP-based rate controls. An attacker can send thousands of requests per second with no enforcement.

Evidence — Request / Response Sample

```
GET https://jsonplaceholder.typicode.com/posts
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Access-Control-Allow-Origin: *

Rate-limit headers observed:
X-RateLimit-Limit:      [NOT PRESENT]
X-RateLimit-Remaining: [NOT PRESENT]
X-RateLimit-Reset:     [NOT PRESENT]
Retry-After:           [NOT PRESENT]

Repeated requests return identical 200 OK with no throttling.
```

Business Impact

Impact Analysis

Absence of rate limiting enables: (1) Credential stuffing attacks against auth endpoints, (2) Enumeration of all user/resource IDs at machine speed, (3) Denial-of-service by overwhelming the server, (4) Automated scraping of the entire dataset. Business impact: service outages, data theft, cloud cost overruns.

Remediation Steps

- 9. Implement per-IP rate limits at the API gateway level (e.g., 100 req/min per IP)
- 10. Return standard 429 Too Many Requests with a Retry-After header when limits are exceeded
- 11. Apply stricter limits on sensitive endpoints (login, password reset, user enumeration)
- 12. Use token bucket or sliding window algorithms for smooth throttling
- 13. Alert on sudden spikes in request volume from a single source

RISK-04 Unauthenticated Write Operations (POST / PUT / DELETE)			HIGH
API Tested	JSONPlaceholder	OWASP Category	API5:2023
Endpoint(s)	POST /posts, PUT /posts/{id}, DELETE /posts/{id}		

Description

State-changing HTTP methods (POST, PUT, PATCH, DELETE) are accepted without any authentication or authorization check. Any anonymous client can 'create' a post, 'update' any resource, or 'delete' any record. While JSONPlaceholder does not persist changes, this demonstrates a critical pattern that, in production, would allow data destruction and unauthorized data creation.

Evidence — Request / Response Sample

```
DELETE https://jsonplaceholder.typicode.com/posts/1
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8

Response: {} ← server accepted deletion with no auth

POST https://jsonplaceholder.typicode.com/posts
Body: { "title": "test", "body": "inject", "userId": 99 }
HTTP/1.1 201 Created
Response: { "id": 101, "title": "test", ... }

Auth token sent: NONE | CSRF token: NONE
```

Business Impact

Impact Analysis

In production: any user can delete any other user's content, create spam/malicious records, or overwrite legitimate data. This represents complete loss of data integrity and can result in regulatory fines, user lawsuits, and reputation damage.

Remediation Steps

- 14. Enforce authentication on ALL write endpoints (POST, PUT, PATCH, DELETE)
- 15. Implement resource ownership checks — users may only modify their own records
- 16. Use CSRF tokens for any session-based authentication
- 17. Apply role-based access control (RBAC): only admins can delete; owners can update
- 18. Log every write operation with user ID, timestamp, and IP address

RISK-05 CORS Wildcard Allows Any Origin			MEDIUM
API Tested	JSONPlaceholder	OWASP Category	API8:2023
Endpoint(s)	All endpoints		

Description

All responses include the header Access-Control-Allow-Origin: * which means any website on the internet can make cross-origin requests to this API from a visitor's browser. While this is intentional for a public test API, the same misconfiguration in a production API with authenticated sessions creates a critical cross-origin attack surface.

Evidence — Request / Response Sample

```
GET https://jsonplaceholder.typicode.com/posts/1
HTTP/1.1 200 OK

Response headers:
  Access-Control-Allow-Origin: *
  Access-Control-Allow-Credentials: [NOT PRESENT]
  Vary: Origin [NOT PRESENT]

Implication: any origin (https://attacker.com) can fetch
this API's data in a victim's browser via JavaScript fetch().
```

Business Impact

Impact Analysis

If a production API uses wildcard CORS alongside cookie-based auth, an attacker can host a malicious page that silently calls the API as the logged-in victim, stealing their data. This is a classic CSRF-variant cross-origin attack.

Remediation Steps

- 19. Never use Access-Control-Allow-Origin: * on authenticated APIs
- 20. Maintain an explicit allowlist of trusted origins (e.g., your own frontend domain)
- 21. Set Vary: Origin header so CDNs don't cache cross-origin responses incorrectly
- 22. If credentials are needed, use Access-Control-Allow-Credentials: true only with explicit origins

RISK-06 Missing Security Headers in All Responses			MEDIUM
API Tested	JSONPlaceholder + ReqRes	OWASP Category	API8:2023
Endpoint(s)	All endpoints		

Description

HTTP responses from both APIs are missing standard security headers that browsers rely on to protect users. The absence of Content-Security-Policy, X-Content-Type-Options, Strict-Transport-Security, and X-Frame-Options leaves the API and any applications built on it exposed to a range of browser-level attacks.

Evidence — Request / Response Sample

Response headers observed (JSONPlaceholder):

Content-Type: application/json; charset=utf-8

Access-Control-Allow-Origin: *

X-Powered-By: Express ← exposes server technology

Via: 1.1 vegur

Headers NOT present:

Strict-Transport-Security: [MISSING]

Content-Security-Policy: [MISSING]

X-Content-Type-Options: [MISSING]

X-Frame-Options: [MISSING]

Referrer-Policy: [MISSING]

Permissions-Policy: [MISSING]

X-Powered-By: Express ← reveals stack (attack intel)

Business Impact

Impact Analysis

Missing HSTS enables SSL stripping attacks. Missing X-Content-Type-Options enables MIME sniffing attacks. Exposing X-Powered-By reveals the tech stack, giving attackers intelligence to target known Express.js vulnerabilities. Collectively reduces the defense-in-depth posture.

Remediation Steps

- 23. Add Strict-Transport-Security: max-age=63072000; includeSubDomains; preload
- 24. Add X-Content-Type-Options: nosniff
- 25. Add X-Frame-Options: DENY or SAMEORIGIN
- 26. Add Referrer-Policy: strict-origin-when-cross-origin

- 27. Remove or spoof the X-Powered-By header to avoid stack fingerprinting
- 28. Use a security header middleware (e.g., Helmet.js for Node/Express)

RISK-07 User Enumeration via Predictable Sequential IDs			MEDIUM
API Tested	JSONPlaceholder + ReqRes	OWASP Category	API1:2023
Endpoint(s)	GET /users/{id}		

Description

User resources are addressed by sequential integer IDs (/users/1, /users/2, ... /users/10). This makes it trivial for an attacker to enumerate all users in the system by simply incrementing the ID parameter. There is no randomization, UUID usage, or access control preventing cross-user data access.

Evidence — Request / Response Sample

```
GET /users/1 → 200 OK { name: 'Leanne Graham', email: '...' }
GET /users/2 → 200 OK { name: 'Ervin Howell', email: '...' }
GET /users/3 → 200 OK { name: 'Clementine Bauch', ... }
...
GET /users/10 → 200 OK { name: 'Clementia DuBuque', ... }
GET /users/11 → 404 Not Found ← reveals total user count

Pattern: purely sequential, no auth, no ownership check.
All 10 users fully enumerated in 11 HTTP requests.
```

Business Impact

Impact Analysis

Attackers can enumerate entire user bases automatically. Combined with unauthenticated access, this allows harvesting of all PII in minutes. The 404 response also leaks the exact total user count — useful intelligence for targeted attacks.

Remediation Steps

- 29. Replace sequential integer IDs with UUIDs (v4) for all user-facing resources
- 30. Validate that the authenticated user has permission to access the requested resource ID
- 31. Return 403 Forbidden (not 404) when a user lacks permission, to avoid confirming existence
- 32. Implement IDOR (Insecure Direct Object Reference) protection at the authorization layer

RISK-08 No Input Validation or Schema Enforcement on POST			LOW
API Tested	JSONPlaceholder	OWASP Category	API6:2023
Endpoint(s)	POST /posts, POST /users		

Description

The API accepts POST requests with arbitrary payloads, including extra fields, incorrect data types, and missing required fields, without returning any validation error. There is no schema enforcement, type checking, or field sanitization visible in the response.

Evidence — Request / Response Sample

```
POST https://jsonplaceholder.typicode.com/posts
Content-Type: application/json

Body sent (with junk fields):
{ "title": 12345,
  "body": null,
  "userId": "not-a-number",
  "injected_field": "<script>alert(1)</script>",
  "extra": { "nested": true } }

Response: HTTP/1.1 201 Created
{ "title": 12345, "body": null, "userId": "not-a-number",
  "injected_field": "<script>alert(1)</script>", "id": 101 }

→ All invalid data accepted and echoed back with no validation.
```

Business Impact

Impact Analysis

In production, lack of input validation enables: SQL/NoSQL injection, XSS payload storage, mass assignment attacks (injecting privileged fields like isAdmin:true), and data corruption. These are consistently in the OWASP Top 10.

Remediation Steps

- 33. Define and enforce a JSON Schema or OpenAPI spec for every request body
- 34. Reject requests with unexpected fields (allowlist approach, not denylist)
- 35. Validate data types, lengths, and formats server-side — never trust client input
- 36. Sanitize string inputs to prevent XSS and injection attacks
- 37. Return descriptive 400 Bad Request errors for invalid payloads

5. Risk Matrix & Priority Remediation

The following matrix maps each finding by likelihood of exploitation versus business impact, enabling security and engineering teams to prioritize remediation efforts effectively.

ID	Severity	Finding	Likelihood	Priority
RISK-01	HIGH	Unauthenticated Endpoint Access	Very High	Immediate
RISK-02	HIGH	Excessive Data Exposure	Very High	Immediate
RISK-03	HIGH	No Rate Limiting	High	Sprint 1
RISK-04	HIGH	Unauthenticated Write Operations	Very High	Immediate
RISK-05	MEDIUM	CORS Wildcard	Medium	Sprint 2
RISK-06	MEDIUM	Missing Security Headers	Medium	Sprint 2
RISK-07	MEDIUM	User ID Enumeration	High	Sprint 1
RISK-08	LOW	No Input Validation	Medium	Sprint 3

6. Strategic Recommendations

6.1 Immediate Actions (Before Next Release)

- Implement authentication on all endpoints using OAuth 2.0 + Bearer tokens
- Add resource-level authorization: verify caller owns the requested resource
- Remove unauthenticated access to write methods (POST, PUT, DELETE)
- Apply field-level response filtering — return only the minimum required data

6.2 Short-Term (Sprint 1–2)

- Deploy API gateway rate limiting: 100 req/min per IP, 429 response with Retry-After
- Replace sequential integer IDs with UUIDs across all resources
- Add Helmet.js (or equivalent) to inject all missing security headers
- Restrict CORS to explicit trusted origin allowlist — remove wildcard

6.3 Long-Term (Architecture Level)

- Adopt API-first design with OpenAPI Specification for every endpoint
- Implement an API gateway (Kong, AWS API Gateway, Apigee) as centralized enforcement point
- Establish automated security scanning in CI/CD pipeline using OWASP ZAP or Spectral
- Create an API inventory and run periodic penetration tests against all consumer-facing APIs
- Train all backend developers on OWASP API Security Top 10 annually

The Developer's API Security Checklist

- ✓ Every endpoint requires authentication (no anonymous access to sensitive data)
- ✓ Every resource access validates caller ownership/authorization
- ✓ Rate limiting enforced at gateway level with 429 responses
- ✓ Response bodies contain only the fields the caller needs
- ✓ All IDs are UUIDs, not sequential integers
- ✓ CORS restricted to known, trusted origins only
- ✓ Security headers present on all responses
- ✓ Input validated against schema on all POST/PUT endpoints
- ✓ Write operations (POST/PUT/DELETE) require explicit authorization check
- ✓ All API interactions are logged with user ID and timestamp

7. Tools & References

Tools Used in This Analysis

Tool	Purpose	URL
Postman	API endpoint testing, header inspection	postman.com
Browser DevTools	Network tab for live header capture	Built into Chrome/Firefox/Edge
MXToolbox	DNS/domain analysis	mxtoolbox.com
OWASP API Top 10	Risk classification framework	owasp.org/API-Security
API Security Checklist	Developer reference	github.com/shieldfy
JSONPlaceholder	Primary test API	jsonplaceholder.typicode.com
ReqRes	Auth simulation test API	reqres.in

8. Conclusion

This API security audit of JSONPlaceholder and ReqRes identified 8 distinct security findings across 3 severity levels. While these are demo APIs designed for testing (not production systems), the patterns and vulnerabilities they exhibit are directly representative of risks found in real-world SaaS API backends.

The most critical findings — unauthenticated endpoint access, excessive data exposure, absence of rate limiting, and unauthenticated write operations — form the foundation of API compromise in real attacks. These four issues alone account for the majority of API breaches reported in industry data.

Organizations that treat API security as an afterthought consistently suffer data breaches that could have been prevented with basic controls. API security is not a nice-to-have — it is a core requirement for any SaaS product handling user data.

Final Note to Development Teams

APIs are your product's front door. Every endpoint you expose is a potential entry point for attackers.

The controls described in this report are not advanced security measures — they are table stakes for any modern API.

Authenticate every request. Authorize every resource. Rate limit every endpoint. Expose only the data you must.

— End of Report —

API Security Risk Analysis | 2025 | Read-Only Audit — Demo APIs Only